DAY-3 Hands On

Pre-Requisites: Before starting with problem solving, please make sure that you have created a database and restored data (refer Sample Database create objects.txt and load data.txt)

Ecommdb – SQL Problem Definitions Based on Advanced Querying and Data Manipulation

Level-1: Problem 1 – Product Analysis Using Nested Queries

Scenario:

You are working as a database developer for an automobile retail company. Management wants to identify products that are priced higher than the average price of products in their respective categories.

📌 Requirements

1. Retrieve product details (product_name, model_year, list_price).

2. Compare each product's price with the average price of products in the same category using a nested query.

3. Display only those products whose price is greater than the category average.

4. Show calculated difference between product price and category average.

5. Concatenate product name and model year as a single column (e.g., 'ProductName (2017)').

🛠️ Technical Constraints

• Use subquery in WHERE clause.

• Use aggregate function (AVG).

• Use string manipulation functions.

• Use arithmetic expressions for price difference calculation.

Expectations:

- Proper use of nested query.
- Correct calculation of average and difference.
- Clean and readable SQL query.

🎯 Learning Outcome

- Understand nested queries with aggregate functions.
- Perform calculations inside SELECT statement.
- Apply string concatenation in SQL.

Level-1: Problem 2 – Customer Activity Classification

Scenario:

The company wants to classify customers based on their total order value and identify customers who have placed orders versus those who have not.

📌 Requirements

1. Use nested query to calculate total order value per customer.
2. Classify customers using conditional logic:
   - 'Premium' if total order value > 10000
   - 'Regular' if total order value between 5000 and 10000
   - 'Basic' if total order value < 5000
3. Use UNION to display customers with orders and customers without orders.
4. Display full name using string concatenation.
5. Handle NULL cases appropriately.

🛠️ Technical Constraints

- Use CASE statement for classification.
- Use UNION operator.
- Use subquery for total calculation.
- Use JOIN between customers and orders tables.

Expectations:

- Proper implementation of UNION.
- Correct usage of CASE expression.
- Accurate total value calculation.

🎯 Learning Outcome

- Apply conditional logic in SQL.
- Combine results using set operators.
- Work with nested aggregation queries.

Level-2: Problem 3 – Store Performance and Stock Validation

Scenario:

Management wants to evaluate store performance by identifying stores that have sold products but currently have zero stock for those products.

📌 Requirements

1. Identify products sold in each store using nested queries.
2. Compare sold products with current stock using INTERSECT and EXCEPT operators.
3. Display store_name, product_name, total quantity sold.
4. Calculate total revenue per product (quantity × list_price – discount).
5. Update stock quantity to 0 for discontinued products (simulation).

🛠️ Technical Constraints

- Use INTERSECT and EXCEPT operators.
- Use subqueries in FROM clause.
- Use arithmetic expressions for revenue calculation.
- Use UPDATE statement for stock modification.

Expectations:

- Correct application of set operators.
- Accurate revenue calculations.
- Proper JOIN between order_items, orders, stores, and stocks tables.

🎯 Learning Outcome

- Master set operators (INTERSECT, EXCEPT).
- Perform advanced nested queries.
- Apply UPDATE statements with conditions.

Level-2: Problem 4 – Order Cleanup and Data Maintenance

Scenario:

The database contains outdated and rejected orders. Management wants to clean up and archive data while maintaining consistency.

📌 Requirements

1. Insert archived records into a new table (archived_orders) using INSERT INTO SELECT.
2. Delete orders where order_status = 3 (Rejected) and older than 1 year.
3. Use nested query to identify customers whose all orders are completed.
4. Display order processing delay (DATEDIFF between shipped_date and order_date).
5. Mark orders as 'Delayed' or 'On Time' using CASE expression based on required_date.

🛠️ Technical Constraints

- Use INSERT statement with SELECT.
- Use DELETE with nested query.
- Use DATE functions and conditional logic.
- Maintain referential integrity constraints.

Expectations:

- Proper handling of DELETE and INSERT operations.
- Correct usage of subqueries for validation.
- Accurate date calculations and CASE logic.

🎯 Learning Outcome

- Perform data modification operations safely.
- Implement business logic using SQL expressions.
- Understand real-world database maintenance scenarios.